

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Tổng quan về Kiểm thử Phần mềm

Buổi 1 / 12 — Môn: Kiểm thử Phần mềm

Ngày 25 tháng 8 năm 2026

Nội dung buổi học

- 1 Giới thiệu môn học: đánh giá, điều kiện dự thi, lộ trình 12 buổi
- 2 Nghề kiểm thử phần mềm — các vị trí công việc
- 3 Kiểm thử phần mềm là gì? Error — Fault — Failure, chi phí lỗi
- 4 Nội dung nghiên cứu của kiểm thử phần mềm
- 5 Bảy nguyên tắc kiểm thử theo ISTQB
- 6 Giới thiệu hệ thống thực hành DigiShield
- 7 Thiết lập môi trường: chạy ứng dụng và chạy bộ test
- 8 Thực hành trên lớp và bài nộp về nhà

Vị trí trong đề cương

Buổi 1 thuộc Chương 1 — Giới thiệu tổng quan

- Khái niệm, mục tiêu và vai trò của kiểm thử phần mềm
- Các nguyên tắc kiểm thử cơ bản (ISTQB)
- Nghề nghiệp trong lĩnh vực kiểm thử
- Làm quen với hệ thống thực hành xuyên suốt môn học: **DigiShield**

Định hướng

Toàn bộ 12 buổi học đều gắn lý thuyết với thực hành trên một codebase thật. Buổi 1 tập trung xây nền tảng khái niệm và **cài đặt thành công môi trường**.

Thông tin môn học

Tổng quan

- Mã học phần: IT1.234.3
- 12 buổi: lý thuyết + thực hành
- Ngôn ngữ thực hành: Java, TypeScript
- Hệ thống thực hành: DigiShield (Spring Boot + React)

Đánh giá học phần (theo đề cương)

- A1.1 Kiểm tra viết giữa kỳ: **10%**
- A1.2 Bài tập lớn: **30%**
- A1.3 Chuyên cần: **10%**
- A2.1 Thi viết cuối kỳ: **50%**

Yêu cầu đầu vào

Đã học lập trình hướng đối tượng (Java); biết Git cơ bản. Kiến thức Spring Boot là lợi thế, không bắt buộc — sẽ được hướng dẫn dần.

Điều kiện dự thi cuối kỳ

Sinh viên được dự thi cuối kỳ khi

- 1 Tham dự $\geq$ **80%** số buổi học trên lớp
- 2 Hoàn thành **80%** số bài tập được giao
- 3 Tham dự **đầy đủ các bài kiểm tra** trên lớp
- 4 Hoàn thành và **bảo vệ bài tập lớn** (buổi 12)

Bài tập lớn (30%) — nhóm 3–5 sinh viên

Kiểm thử một **module của DigiShield** (hoặc ứng dụng nhóm tự chọn được GV duyệt): lập test plan $\rightarrow$ thiết kế test case hộp đen + hộp trắng $\rightarrow$ cài đặt JUnit/Mockito $\rightarrow$ API test Postman/Newman $\rightarrow$ Selenium E2E $\rightarrow$ báo cáo + demo ở buổi 12. Giao đề tài **ngay hôm nay**; hướng dẫn chi tiết ở **Buổi 2**.

Lộ trình 12 buổi (1/2)

Buổi	Chương	Chủ đề
1	C1	Tổng quan KTPM, 7 nguyên tắc, nghề tester, setup DigiShield
2	C2	SDLC/V-model/Agile, STLC, mức kiểm thử, Test Plan, giao BTL
3	C3	Hộp trắng: CFG, độ phủ câu lệnh / nhánh / đường
4	C3	Độ phủ điều kiện / MC-DC, cyclomatic, luồng dữ liệu, JaCoCo
5	C4	JUnit 5 cơ bản: lifecycle, assertions, AssertJ, parameterized
6	C4	Mockito: mock / stub / verify / captor, test exception

Ghi chú: C1–C7 là các chương của đề cương học phần. Mỗi buổi đều có phần thực hành trực tiếp trên mã nguồn DigiShield.

Lộ trình 12 buổi (2/2)

Buổi	Chương	Chủ đề
7	C4	Integration test: @SpringBootTest, MockMvc, Testcontainers
8	C5	Hộp đen: EP, BVA, bảng quyết định, chuyển trạng thái, use case
9	C5	Khuôn mẫu đặc tả test case; API testing: Postman + Newman
10	C6	Selenium 1: WebDriver, locators, thao tác form, test đăng nhập
11	C6	Selenium 2: waits, Page Object Model, kịch bản E2E, headless
12	C7	Smoke/regression/security/performance, kiểm thử hệ AI, CI/CD; tổng kết + bảo vệ BTL

Mốc quan trọng: Buổi 2: nhận đề bài tập lớn — Buổi 12: bảo vệ bài tập lớn.

Các vị trí công việc trong Kiểm thử

Nhóm kiểm thử thủ công / nghiệp vụ

- **Manual Tester / QA Tester:** thiết kế và thực thi test case
- **QA Analyst:** phân tích yêu cầu, rủi ro, tiêu chí chấp nhận
- **Test Lead:** lập kế hoạch, phân công, theo dõi tiến độ kiểm thử

Nhóm kỹ thuật / tự động hóa

- **Automation Tester / SDET:** viết test tự động API, UI, integration
- **Performance Tester:** load/stress test, tìm bottleneck
- **QA Engineer trong DevOps:** tích hợp test vào CI/CD pipeline

Năng lực cốt lõi

Tư duy phân tích, hiểu nghiệp vụ, kỹ năng viết test case, lập trình cơ bản, dùng công cụ kiểm thử và giao tiếp tốt với developer/product owner.

Con đường phát triển nghề nghiệp



Chứng chỉ phổ biến:

- ISTQB Foundation / Advanced [4]
- Chứng chỉ công cụ: Selenium, Postman

Xu hướng thị trường:

- Ưu tiên tester biết lập trình (SDET)
- Kiểm thử bảo mật, kiểm thử hệ thống AI

IEEE 610.12-1990 [7]

Kiểm thử phần mềm là quá trình **vận hành** một hệ thống hoặc thành phần trong các điều kiện được chỉ định, **quan sát** và **ghi lại kết quả**, và **đánh giá** một số khía cạnh của hệ thống hoặc thành phần đó.

Kiểm thử **KHÔNG** phải là:

- Chứng minh phần mềm đúng
- Đảm bảo không còn lỗi
- Chỉ chạy chương trình xem có lỗi không

Kiểm thử **LÀ**:

- Tìm kiếm lỗi một cách có hệ thống
- Đánh giá chất lượng phần mềm
- Cung cấp thông tin để ra quyết định

Mục tiêu của kiểm thử

Đánh giá chất lượng

- Đo mức độ đáp ứng yêu cầu chức năng và phi chức năng
- Cung cấp thông tin cho quyết định phát hành
- Xây dựng niềm tin vào sản phẩm

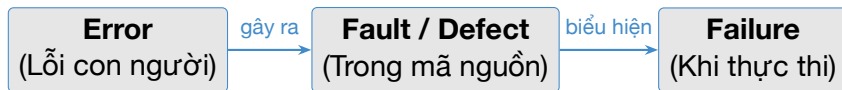
Phát hiện lỗi

- Tìm khiếm khuyết (defect) **trước** người dùng cuối
- Ngăn lỗi lan sang giai đoạn sau
- Giảm rủi ro vận hành và chi phí sửa chữa

Quan điểm của Myers (The Art of Software Testing) [3]

Một test case **tốt** là test case có xác suất cao **tìm ra một lỗi chưa được phát hiện** — không phải test case chứng tỏ chương trình chạy đúng.

Error — Fault (Defect) — Failure



Ví dụ trên DigiShield — phân trang nhật ký can thiệp

- **Error:** lập trình viên *quên* rằng tham số size do client gửi lên cần được giới hạn (clamp)
- **Fault:** `listInterventions(page, size)` dùng thẳng size để truy vấn, không chặn trên 100
- **Failure:** client gọi `GET /interventions?size=1000` → hệ thống trả về 1000 bản ghi, quá tải bộ nhớ và băng thông

Mã thật của DigiShield đã “vá” fault đó thế nào?

modules/interception/.../InterceptionServiceImpl.java

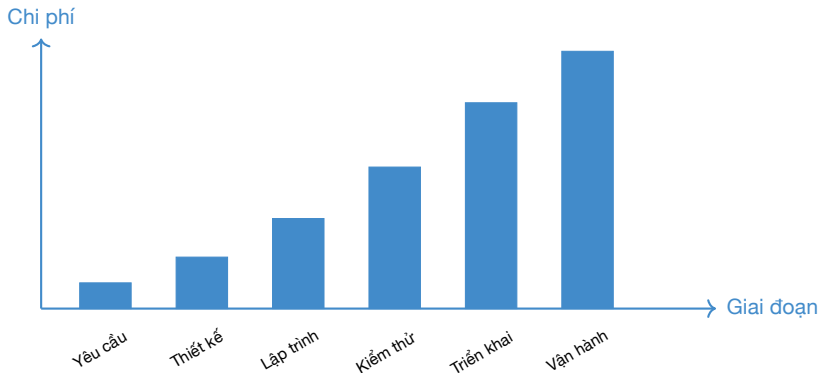
```
public List<InterventionEventView> listInterventions(  
    int page, int size) {  
    // Chan duoi: page toi thieu la 1  
    int safePage = Math.max(page, 1);  
    // Chan hai dau: 1 <= size <= 100  
    int safeSize = Math.min(Math.max(size, 1), 100);  
    // ... truy van voi safePage, safeSize  
}
```

Câu hỏi kiểm thử: những giá trị size nào đáng để thử? 0, 1, 2, 99, 100, 101? —
Đây chính là **phân tích giá trị biên (BVA)**, sẽ học ở Buổi 8.

Tại sao cần kiểm thử? — Chi phí lỗi

Quy tắc 1 : 10 : 100

Chi phí sửa lỗi tăng theo cấp bậc khi phát hiện muộn hơn trong vòng đời.



Nguồn: Boehm & Papaccio [6], NIST [8]

Chi phí lỗi — Ví dụ lịch sử

Ariane 5 (1996)

Tên lửa nổ 37 giây sau khi phóng do lỗi tràn số (overflow) khi chuyển đổi float64 sang int16. Thiệt hại: \$370 triệu.

Knight Capital (2012)

Lỗi phần mềm giao dịch tự động gây thua lỗ \$440 triệu trong 45 phút.

Boeing 737 MAX MCAS (2018–2019)

Lỗi logic trong hệ thống kiểm soát bay không được kiểm thử đủ trong điều kiện lỗi cảm biến. Hậu quả: 346 người tử vong, thiệt hại hàng chục tỷ USD.

Bài học: kiểm thử không đầy đủ không chỉ tốn kém về tài chính mà còn gây hậu quả không thể đảo ngược.

Bối cảnh Việt Nam — động lực của DigiShield

Thực trạng đáng lo ngại:

- Hàng trăm nghìn phản ánh lừa đảo trực tuyến được ghi nhận mỗi năm (Cục An toàn thông tin)
- Kịch bản phổ biến: giả danh công an / ngân hàng, tin nhắn brandname giả, đường link phishing, dụ chuyển tiền cho “người quen”
- Nạn nhân thường chuyển tiền cho **tài khoản mới thêm** ngay **trong lúc đang nghe điện thoại** kẻ lừa đảo

Hệ quả cho môn học

Phần mềm chống lừa đảo là hệ thống **nhạy cảm về an toàn**: một quyết định sai (cho phép giao dịch đáng ngờ, hoặc chặn nhầm giao dịch hợp lệ) đều gây thiệt hại thật. $\Rightarrow$ kiểm thử kỹ lưỡng là **bắt buộc**, không phải tùy chọn.

Verification và Validation

Verification — Xác minh

*“Are we building the product **right**?”*

- Sản phẩm có đúng với **đặc tả** không?
- Review tài liệu, thiết kế, mã nguồn
- Thực hiện được từ rất sớm

Validation — Thẩm định

*“Are we building the **right** product?”*

- Sản phẩm có đáp ứng **nhu cầu người dùng** không?
- Chạy thử hệ thống trong bối cảnh thật
- Thường thực hiện ở giai đoạn sau

Ví dụ DigiShield

Verification: quyết định can thiệp có đúng quy tắc đặc tả (đang nghe điện thoại + người nhận mới + tài khoản trong danh sách theo dõi → tạm dừng)?

Validation: màn hình cảnh báo có thực sự **ngăn được** người dùng lớn tuổi chuyển tiền cho kẻ lừa đảo hay không?

Kiểm thử tĩnh và kiểm thử động

Kiểm thử tĩnh (Static)

Không thực thi chương trình:

- Review yêu cầu, thiết kế, test case
- Code review, walkthrough, inspection
- Phân tích tĩnh bằng công cụ (linter, SonarQube)

Kiểm thử động (Dynamic)

Có thực thi chương trình:

- Unit test, integration test, system test
- Kiểm thử hộp đen / hộp trắng
- Kiểm thử hiệu năng, bảo mật

Lưu ý

Kiểm thử tĩnh tìm **fault trực tiếp trong sản phẩm trung gian** (work product); kiểm thử động quan sát **failure** rồi truy ngược về fault. Hai loại bổ sung cho nhau, không thay thế nhau.

Đối tượng của kiểm thử

Đối tượng	Ví dụ trên DigiShield	Kỹ thuật
Tài liệu yêu cầu	Đặc tả quy tắc chấm điểm rủi ro	Review
Thiết kế / kiến trúc	Ranh giới 9 module Spring Modulith	Review, ArchUnit
Mã nguồn (đơn vị)	<code>InterceptionServiceImpl.evaluate()</code>	Unit test
Tích hợp các thành phần	Sự kiện giữa module ai → analytics	Integration test
API / dịch vụ	<code>POST /api/v1/interventions/evaluate</code>	API test
Giao diện người dùng	Trang đăng nhập, SOC Inbox (React)	E2E / Selenium
Thuộc tính phi chức năng	Hiệu năng, bảo mật đa tenant	Perf / security

Môn học sẽ lần lượt đi qua tất cả các đối tượng này trong 12 buổi.

Kiểm thử hộp đen và hộp trắng — nhìn trước

Hộp đen (Black-box)

- Chỉ nhìn **đầu vào / đầu ra**, không nhìn mã nguồn
- Dựa trên đặc tả và yêu cầu
- Kỹ thuật: EP, BVA, bảng quyết định... (Buổi 8–9)

Hộp trắng (White-box)

- Nhìn vào **cấu trúc bên trong** của mã nguồn
- Dựa trên luồng điều khiển, luồng dữ liệu
- Kỹ thuật: độ phủ câu lệnh / nhánh / đường (Buổi 3–4)

Cùng một hàm — hai góc nhìn [2]

Hàm `evaluate()` của DigiShield: hộp đen xem “giao dịch nào bị tạm dừng?” theo đặc tả nghiệp vụ; hộp trắng xem “mọi nhánh `if` đã được chạy qua chưa?”.

Tổng quan 7 nguyên tắc ISTQB

- 1 Kiểm thử cho thấy sự **hiện diện** của lỗi, không phải sự **vắng mặt**
- 2 Kiểm thử **vét cạn** (exhaustive testing) là bất khả thi
- 3 Kiểm thử **sớm** (early testing) tiết kiệm chi phí
- 4 **Phân cụm** lỗi (defect clustering)
- 5 **Nghịch lý thuốc trừ sâu** (pesticide paradox)
- 6 Kiểm thử phụ thuộc vào **ngữ cảnh** (context-dependent)
- 7 **Ngụy biện** “không có lỗi” (absence-of-errors fallacy)

Lưu ý

Các nguyên tắc này không phải quy tắc cứng nhắc mà là **hướng dẫn tư duy** cho tester khi ra quyết định (theo ISTQB Foundation Syllabus [5]).

Nguyên tắc 1: Kiểm thử cho thấy sự hiện diện của lỗi

Phát biểu

Kiểm thử có thể chứng minh phần mềm **có lỗi**, nhưng **không thể** chứng minh phần mềm hoàn toàn không có lỗi.

Hệ quả thực tế:

- Khi tất cả test case pass, không có nghĩa phần mềm đúng
- Cần liên tục thiết kế test case mới
- Mục tiêu: *giảm rủi ro*, không phải *đảm bảo hoàn hảo*

Ví dụ DigiShield

InterceptionServiceImpl hiện có 5 unit test đều pass — nhưng điều đó không chứng minh logic can thiệp đúng với **mọi** tổ hợp tín hiệu, số tiền và trạng thái cuộc gọi có thể xảy ra.

Nguyên tắc 2: Kiểm thử vét cạn là bất khả thi

Vấn đề

Số tổ hợp đầu vào của bất kỳ hệ thống thực tế (non-trivial) nào cũng **vô hạn hoặc quá lớn** — không thể kiểm thử hết mọi trường hợp.

Ví dụ định lượng — DigiShield

API POST /api/v1/interventions/evaluate nhận:

- amount: số thập phân (BigDecimal) — miền giá trị khổng lồ
- destAccount: chuỗi số tài khoản bất kỳ
- onCall, newPayee: 2 boolean — $2 \times 2 = 4$ tổ hợp

Chỉ riêng amount và destAccount đã cho số tổ hợp $> 10^{50}$ — không thể thử hết.

Giải pháp: ưu tiên test case theo **rủi ro** và **tầm quan trọng nghiệp vụ** (kỹ thuật EP, BVA — Buổi 8).

Nguyên tắc 3 & 4: Kiểm thử sớm và Phân cụm lỗi

Nguyên tắc 3: Kiểm thử sớm

- Bắt đầu kiểm thử ngay từ giai đoạn phân tích yêu cầu
- Review đặc tả, review thiết kế — đây là kiểm thử tĩnh
- Chi phí sửa lỗi tăng theo cấp bậc khi phát hiện muộn

Nguyên tắc 4: Phân cụm lỗi (defect clustering)

Nguyên lý Pareto (80/20): phần lớn lỗi tập trung ở **một số ít module**.

DigiShield

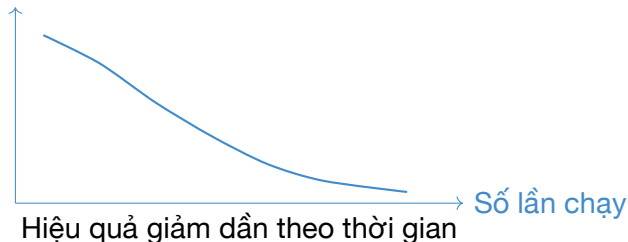
Module interception và analytics chứa nhiều logic điều kiện, biên số học nhất → đáng được ưu tiên kiểm thử hơn module tenancy chủ yếu CRUD.

Nguyên tắc 5: Nghịch lý thuốc trừ sâu

Phát biểu (pesticide paradox)

Nếu cùng một tập test case được chạy lặp đi lặp lại, chúng sẽ **mất khả năng phát hiện lỗi mới** theo thời gian.

Lỗi tìm được



Giải pháp: Định kỳ **xem xét và bổ sung** test case mới. Ví dụ DigiShield: kẻ lừa đảo đổi chiêu thức liên tục → bộ test cho StubAiClient (phân loại threat/spam/clean) phải được cập nhật theo mẫu lừa đảo mới.

Nguyên tắc 6 & 7

Nguyên tắc 6: Phụ thuộc ngữ cảnh

Không có cách tiếp cận kiểm thử duy nhất cho mọi dự án.

Hệ thống

Ưu tiên

Y tế	Safety, reliability
Chống lừa đảo	Security, accuracy
Game	Gameplay, trải nghiệm chơi
E-commerce	Performance, UX

Nguyên tắc 7: Ngụy biện “không có lỗi”

Absence-of-errors fallacy: một hệ thống vượt qua toàn bộ test case nhưng **không đáp ứng nhu cầu người dùng** vẫn là hệ thống thất bại.

Ví dụ DigiShield

Màn hình cảnh báo giao dịch hiển thị đúng 100% test case, nhưng nội dung khó hiểu với người lớn tuổi → họ vẫn bấm “Tiếp tục” và mất tiền.

DigiShield — “Lá Chắn Sốt”

Bài toán

Nền tảng **chống lừa đảo / phishing** cho thị trường Việt Nam:

- **Đào tạo** nhận thức an toàn thông tin cho nhân viên tổ chức
- **Mô phỏng** tấn công phishing để đánh giá mức độ cảnh giác
- **Tiếp nhận báo cáo** lừa đảo và phân loại bằng AI
- **Chấm điểm rủi ro** người dùng / phòng ban / tổ chức
- **Can thiệp giao dịch** đáng ngờ theo thời gian thực

Vì sao dùng DigiShield để học kiểm thử?

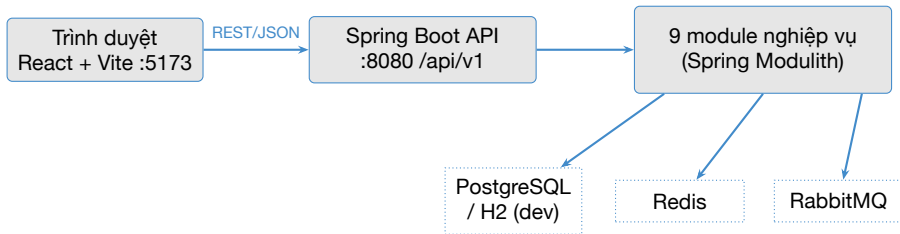
Codebase thật, nghiệp vụ nhạy cảm về an toàn, có sẵn 398 test tự động làm mẫu, đủ “đắt” cho mọi kỹ thuật: unit, integration, API, E2E.

Chín module nghiệp vụ

Module	Chức năng
auth	Đăng nhập, phân quyền, MFA, quản lý người dùng
tenancy	Quản lý tổ chức (tenant), cách ly dữ liệu đa tenant
learning	Khóa học, quiz đào tạo nhận thức an toàn
simulation	Chiến dịch mô phỏng tấn công phishing
reporting	Tiếp nhận và xử lý (triage) báo cáo lừa đảo
analytics	Chấm điểm rủi ro, dashboard, benchmark
notification	Thông báo, cảnh báo broadcast, WebSocket
ai	Phân loại nội dung, sinh mẫu phishing (Claude API)
interception	Đánh giá và can thiệp giao dịch đáng ngờ, watchlist

Mỗi nhóm BTL sẽ chọn **một module** làm đối tượng kiểm thử xuyên suốt.

Kiến trúc tổng thể



- **Backend:** Java 25, Spring Boot 4.1, Spring Modulith 2.1 — một ứng dụng (monolith) chia 9 module ranh giới rõ ràng, Gradle đa module
- Giao tiếp giữa module bằng **sự kiện** nội bộ; đa tenant bằng PostgreSQL Row-Level Security; WebSocket cho thông báo

Cấu trúc mã nguồn

Bố cục thư mục backend

- `boot/app` — ứng dụng khởi chạy, cấu hình, test tích hợp
- `modules/auth`, `modules/interception`,... — 9 module nghiệp vụ, mỗi module có `src/main` và `src/test` riêng
- `deploy/` — Docker Compose, Helm chart
- `../docs/DigiShield_openapi.yaml` — đặc tả OpenAPI (ở thư mục cha, ngoài repo)

Frontend

- React 18 + TypeScript + Vite 5
- TanStack Query + axios; client API sinh tự động từ OpenAPI (Orval)
- Unit test frontend: Vitest
- **Chưa có E2E** — phần Selenium do sinh viên bổ sung (Buổi 10–11)

Nhìn thử một class sẽ gặp lại nhiều lần

InterceptionServiceImpl.evaluate() (rút gọn)

```
public InterventionDecision evaluate(EvaluateRequest req) {  
    boolean watchlistHit = watchlist.contains(  
        req.destAccount());  
    // Quy tắc nghiệp vụ chống lừa đảo  
    if (req.onCall() && req.newPayee() && watchlistHit) {  
        return pause(req);    // Tam dung giao dịch  
    } else if (watchlistHit) {  
        return warn(req);    // Canh bao nguoi dung  
    }  
    return allow(req);    // Cho phép  
}
```

Ba đầu ra PAUSE / WARN / ALLOW — đây là ví dụ “chủ lực” xuyên suốt môn học: đồ thị luồng điều khiển (Buổi 3), bảng quyết định (Buổi 8), unit test với Mockito (Buổi 6), API test (Buổi 9).

Bộ test có sẵn của DigiShield

Hai tầng test (JVM Test Suites)

- `./gradlew test` — **unit test** thuần (*Test.java), không cần Docker, chạy nhanh
- `./gradlew integrationTest` — test tích hợp (*IT.java), PostgreSQL thật qua Testcontainers, cần Docker

Con số hiện tại

- **398 unit test** / 50 class test
- JUnit 5 + Mockito + AssertJ
- JaCoCo đo độ phủ (Buổi 4)
- ModularityTests kiểm tra ranh giới module

Ý nghĩa với môn học

Sinh viên **đọc test có sẵn để học cách viết**, rồi **bổ sung** test mới cho các trường hợp còn thiếu — đúng cách một tester gia nhập dự án thật.

Yêu cầu môi trường

Bắt buộc

- JDK 25 (Eclipse Temurin)
- Node.js 20+ và npm
- Git 2.x+
- IDE: IntelliJ IDEA hoặc VS Code

Tùy chọn (cần từ Buổi 7)

- Docker Desktop — cho Testcontainers và chế độ prod-like
- Postman — cho API testing (Buổi 9)

Phần cứng khuyến nghị

- RAM: 8 GB trở lên (16 GB nếu dùng Docker)
- Ổ đĩa: 10 GB trống
- OS: macOS, Linux, hoặc Windows 10/11

Tin vui

Chế độ dev dùng **H2 in-memory**: chạy backend **không cần Docker**, không cần cài cơ sở dữ liệu.

Chạy backend ở chế độ dev

Bước 1: Clone repository (theo link GV cung cấp)

```
git clone <repo-digishield>  
cd digishield/digishield    # backend nam trong repo
```

Repo chứa digishield/ (backend), frontend/ và docs/. Chỉ trong digishield/ mới có gradlew — đứng ở thư mục gốc của repo thì lệnh ./gradlew sẽ báo không tìm thấy.

Bước 2: Khởi động backend với profile dev

```
./gradlew :boot:app:bootRun \  
--args='--spring.profiles.active=dev'
```

- H2 in-memory tự tạo schema, **dữ liệu demo tự seed**
- API sẵn sàng tại `http://localhost:8080/api/v1`
- H2 console (xem dữ liệu): `http://localhost:8080/h2-console`
- Lần chạy đầu Gradle tải dependency, mất 3–5 phút

Chạy frontend và đăng nhập demo

Khởi động frontend (frontend/ ngang hàng với digishield/)

```
cd ../frontend # tu trong repo digishield
npm install    # lan dau tien
npm run dev    # mo http://localhost:5173
```

Đăng nhập dev

Chọn **Vai trò** rồi bấm **Đăng nhập (dev)**.
Không có ô mật khẩu và không kiểm tra gì
cả — Cognito chưa cấu hình.

Vai trò (roles)

SUPER_ADMIN, ORG_ADMIN,
MANAGER, CONTENT_EDITOR,
ANALYST, LEARNER (+
TENANT_ADMIN — alias cũ của
ORG_ADMIN).

Tài khoản seed (admin@, analyst@, learner@coquan.gov.vn...) là **dữ liệu demo** ở màn Người dùng, không phải thông tin đăng nhập. Đăng nhập xong hãy dạo quanh: Dashboard, SOC Inbox, Watchlist, Campaigns — các trang sẽ kiểm thử bằng Selenium ở Buổi 10–11.

Chạy bộ test có sẵn

Lệnh chạy toàn bộ unit test (không cần Docker)

```
./gradlew test
```

Kết quả mong đợi (rút gọn)

```
> Task :modules:interception:test
> Task :modules:analytics:test
> Task :modules:ai:test
> Task :boot:app:test
```

```
BUILD SUCCESSFUL in 32s
80 actionable tasks: 80 executed
```

Chạy thành công thì **không in ra số test** — Gradle chỉ in `N tests completed`, `X failed` khi có test hỏng. Muốn xem số lượng và từng test case, mở `build/reports/tests/test/index.html` trong từng module (397 test ở thời điểm biên soạn).

Đọc thử một test thật

InterceptionServiceImplTest (rút gọn)

```
@Test
void pausesWhenOnCallNewPayeeAndWatchlisted() {
    // Arrange: tài khoản nam trong danh sách theo dõi
    givenWatchlistContains("0123456789");
    var req = new EvaluateRequest(userId,
        new BigDecimal("5000000"), "0123456789",
        true, true); // onCall=true, newPayee=true
    // Act
    var decision = service.evaluate(req);
    // Assert
    assertThat(decision.decision()).isEqualTo("pause");
}
```

Cấu trúc **Arrange – Act – Assert** sẽ được dạy kỹ ở Buổi 5. Hôm nay chỉ cần **chạy được và đọc hiểu đại ý**.

Xử lý sự cố thường gặp khi cài đặt

Triệu chứng	Cách xử lý
Unsupported class file major version	Sai phiên bản JDK — cài JDK 25, kiểm tra <code>java -version</code> và <code>JAVA_HOME</code>
Port 8080 already in use	Tắt tiến trình đang chiếm cổng, hoặc đổi cổng: <code>--args='--spring.profiles.active=dev --server.port=8081'</code>
<code>npm install</code> lỗi mạng	Dùng mạng khác / cấu hình lại registry npm
Trang localhost:5173 trắng	Kiểm tra backend 8080 đã chạy chưa; xem Console của trình duyệt
<code>gradlew: Permission denied</code>	<code>chmod +x gradlew</code> (macOS/Linux)

Quy tắc lớp học

Gặp lỗi lạ: chụp **nguyên văn thông báo lỗi** và hỏi trên nhóm lớp — mô tả lỗi chính xác cũng là một kỹ năng của tester.

Bài tập trên lớp (60 phút)

Phần 1: Cài đặt và chạy ứng dụng (35 phút)

- 1 Cài JDK 25, Node 20+, clone repo DigiShield
- 2 Chạy backend profile dev, xác nhận `localhost:8080/api/v1`
- 3 Chạy frontend, dev sign-in: chọn vai trò rồi bấm Đăng nhập (không có ô mật khẩu)
- 4 Dạo quanh ứng dụng: Dashboard, SOC Inbox, Watchlist

Phần 2: Chạy và quan sát bộ test (25 phút)

- 1 Chạy `./gradlew test`, xác nhận **BUILD SUCCESSFUL**
- 2 Mở báo cáo HTML của module `interception`: **đếm số test pass** và liệt kê 5 test của `InterceptionServiceImplTest`
- 3 Thử **làm hỏng** một test: sửa tạm một assertion, chạy lại, quan sát báo cáo fail, rồi hoàn tác (`git checkout .`)

Ghi chú: `npm run dev` không đặt `VITE_COGNITO_*` nên /login hiện dev sign-in form (bản deploy thì qua Cognito). Làm việc thẳng trên main.

Câu hỏi thảo luận nhanh

- 1 Trong bài thực hành, khi bạn sửa assertion và test fail: fail đó là **failure của phần mềm** hay **fault của test**?
- 2 Nếu 398 test đều pass, ta kết luận được gì — và **không** kết luận được gì — về chất lượng DigiShield? (Nguyên tắc 1)
- 3 Module nào của DigiShield bạn dự đoán chứa nhiều lỗi tiềm ẩn nhất? Vì sao? (Nguyên tắc 4)
- 4 Vì sao `./gradlew test` chạy được mà không cần Docker, còn `integrationTest` thì cần?

Gợi ý

Không có đáp án duy nhất — quan trọng là **lập luận** dựa trên khái niệm đã học: error/fault/failure, nguyên tắc ISTQB, phân tầng test.

Bài nộp — Deadline: trước Buổi 2

Yêu cầu nộp: Báo cáo cài đặt thành công

1 Ảnh chụp màn hình ứng dụng đang chạy:

- Frontend sau khi đăng nhập (thấy Dashboard, có URL `localhost:5173`)
- Terminal backend cho thấy Spring Boot đã khởi động

2 Ảnh chụp kết quả `./gradlew test`:

- Thấy dòng `BUILD SUCCESSFUL` và tổng số test
- Ghi rõ: bao nhiêu test pass trên máy của bạn?

3 Trả lời ngắn (3–5 câu): kể tên 9 module của DigiShield và chọn 1 module bạn muốn làm BTL, kèm lý do

Định dạng nộp

File PDF, đặt tên: `HoTen_MaSV_Buoi01.pdf`. Chuẩn bị cho Buổi 2: **đọc trước Chương 2** của đề cương (quy trình kiểm thử, SDLC, STLC, mô hình V).

Câu hỏi ôn tập

- 1 Phát biểu Nguyên tắc 1 của ISTQB. Cho ví dụ cụ thể trên DigiShield.
- 2 Tại sao kiểm thử vét cạn (exhaustive testing) là bất khả thi? Giải pháp thay thế là gì?
- 3 Phân biệt **Error**, **Fault** và **Failure**. Lấy ví dụ với hàm `listInterventions(page, size)`.
- 4 Phân biệt Verification và Validation. Cho ví dụ trên tính năng can thiệp giao dịch.
- 5 Kiểm thử tĩnh khác kiểm thử động ở điểm nào? Mỗi loại cho một ví dụ.
- 6 DigiShield có mấy module nghiệp vụ? Lệnh nào chạy bộ unit test có sẵn và hiện có bao nhiêu test?

Tóm tắt Buổi 1

Lý thuyết đã học:

- Định nghĩa và mục tiêu kiểm thử phần mềm
- Error — Fault — Failure
- Chi phí lỗi: quy tắc 1:10:100
- Verification / Validation, tĩnh / động
- 7 nguyên tắc ISTQB
- Nghề nghiệp trong kiểm thử

Thực hành đã làm:

- Chạy backend DigiShield (profile dev, H2)
- Chạy frontend, đăng nhập demo
- Chạy `./gradlew test` — 398 test
- Đọc test thật, làm hỏng và sửa lại một test

Buổi tới (Buổi 2 — Chương 2)

Quy trình kiểm thử: SDLC, V-model, Agile, STLC, mức kiểm thử, Test Plan — và **giao đề bài tập lớn**.

Tài liệu tham khảo

- [1] Slide bài giảng điện tử.
- [2] Paul Ammann & Jeff Offutt. *Introduction to Software Testing*. Cambridge University Press, 2008.
- [3] Glenford J. Myers. *The Art of Software Testing*, 2nd edition. John Wiley & Sons, 2004.
- [4] ISTQB — International Software Testing Qualifications Board.
<https://istqb.org/>
- [5] ISTQB Foundation Level Syllabus, version 4.0, 2023.
- [6] Boehm, B. W. and Papaccio, P. N. “Understanding and Controlling Software Costs”. *IEEE Trans. Software Engineering*, 1988.
- [7] IEEE Std 610.12-1990. *IEEE Standard Glossary of Software Engineering Terminology*. IEEE, 1990.
- [8] NIST. *The Economic Impacts of Inadequate Infrastructure for Software Testing*. Planning Report 02-3, 2002.